FINAL-YEAR PROJECT · CITY UNIVERSITY OF HONG KONG · DEPARTMENT OF ELECTRICAL ENGINEERING

Reproducing MIFlu and Benchmarking Influenza Forecasting

Internal technical report — reproduction, 16-arm benchmark, peak skill, and 11 frozen LLM encoders

Document	Internal technical report (long form)
Project	Independent reproduction of MIFlu and a three-way influenza-forecasting benchmark
Version / date	1.0 — 2 November 2026
Audience	Supervisor, assessor, and the next agent (CodeBuddy / ChatGPT) who continues the work
Status	COMPLETE: reproduction done (+4.5%, sign crossover 2-2); benchmark done (28 models).
Companion documents	progress_report_en.docx (short, school template) · ieee_manuscript_en.docx (prototype) · MANIFEST.md (file inventory) · 04_详细说明文档/REPO_HYGIENE.md
Rule	Every number below is read from `02_数字表格/*.csv` at build time and asserted by `dlv/06_原始数据集.py`. Nothing is quoted from memory.

Contents

1. Data
2. Methodology
3. Reproduction of MIFlu's Table V
4. Benchmark 1 benchmark
5. Benchmark 2, and the central finding
6. Benchmark 3 against efficiency
8. Frozen LLM encoders as feature extractors
16. How to reproduce this report
11. References

1. Data

CDC national influenza data: 966 weekly rows x 7 channels, bit-identical to the author's file. Split: 676/97/193 (70/10/20), time-ordered. Normalised with StandardScaler on train only.

Why the 676/97/193 (70/10/20) split? This is the standard protocol of the LTSF benchmark family (Informer, AAAI 2023; DLinear, AAAI 2023; PatchTST, ICLR 2023; iTransformer, ICLR 2024), used by the original MIFlu repository. The integer arithmetic is: 966 x 0.7 = 676 train, 966 x 0.2 = 193 test, remainder 97 validation. We follow this exactly for comparability.

Why (W//16)*16 evaluation windows? This is NOT a journal requirement — it is a side effect of the original code's DataLoader (batch_size=16, drop_last=True discards the last incomplete batch of test windows). We adopt it for comparability with the paper. The discarded windows (e.g., 10 of 170 at L=24) are approximately 1.41x harder than the retained ones.

2. Methodology

Two protocols, never mixed. (1) Reproduction: 10 seeds (2021-2030), validation-selected checkpoint, 7-variable z-space MSE — for comparing with the paper's Table V. (2) Benchmark: 3 seeds (42/43/44), shared windows (W//16)*16 = 160/144/144/128 — for comparing 28 models. Two deliberate deviations [DEVIATION]: checkpoint selection (validation vs test = data leakage fix) and 7-variable metric (fixing test-loader bug). Eval windows: NOT a journal requirement — side effect of batch_size=16, drop_last=True.

Reproduction: ours vs original paper/repo.

Parameter	Ours	Original	Note
Data	966 weeks x 7 vars, CDC	Same	Byte-identical (SHA-256)
Split	676/97/193 (70/10/20)	Same	Time-ordered
Input window T	104 weeks	Same	
Horizon L	24/36/48/60	Same	Trained independently
Seeds (repro)	2021-2030 (10 reps)	fix_seed=2021 (1)	[DEVIATION] Paper claims 10 reps
Epochs	20	default=1	1 epoch does not converge
LR	1e-4	code 1e-4 / paper 5e-4	Paper self-contradictory
Batch size	16	Same	drop_last=True
Optimizer	Adam	Same	
GPT-2 layers/d_model	6/768	Same	
LoRA	r=4, a=4, drop=0.05, c_attn	Same	
Trainable params	1,673,496	Same	LoRA+LN+head
Checkpoint selection	VALIDATION MSE	TEST MSE	[DEVIATION] Test= data leakage
Metric	7-var aggregate MSE/MAE	Same (bug: 1 var)	[DEVIATION] Fix bug
Eval windows	(W//16)*16 = 160/144/144/128	Same (drop_last side effect)	Not a journal requirement

Two [DEVIATION] items are required by top-tier journal standards.

What are the two [DEVIATION] items and why are they necessary? (1) **Checkpoint selection**: the original code selects the best epoch using the TEST set MSE — this is data leakage (the test set is the final exam; you must not consult it when choosing your model). Top-tier venues (NeurIPS, ICML, ICLR) all prohibit this. We use the VALIDATION set. (2) **7-variable metric**: the original code's test loader covers only 1 of 7 variables (a bug); Table V requires 7-variable aggregate MSE. We fix this bug.

3. Reproduction of MIFlu's Table V

Table 3 is the reproduction. The four-horizon mean is **1.4995 against the paper's 1.4355 — 4.5 % worse** — but the mean is the least interesting part of it. The sign flips with the horizon: at L=24 we are 15.4 % better than the paper (1.3045 against 1.542) and at L=36 4.6 % better (1.3561 against 1.422), while at L=48 we are 17.0 % worse (1.6543 against 1.414) and at L=60 23.4 % worse (1.6829 against 1.364). The MAE comparison shows the

same sign crossover: L24 0.7482 vs 0.726 (+3.1%), L36 0.7712 vs 0.779 (-1.0%), L48 0.8381 vs 0.757 (+10.7%), L60 0.8596 vs 0.719 (+19.6%); 4-horizon mean 0.8043 vs 0.7452 (+7.9%).

Table 3. Reproduction of MIFlu's Table V — the sign crossover. 7-variable aggregate MSE in z-space, 10 repetitions, seeds 2021–2030.

Horizon	Ours (10-rep mean ± sd)	Paper	Δ MSE	Sign	Ours MAE	Paper MAE	Δ MAE
L=24	1.3045 ± 0.0883	1.542	-15.4 %	**WIN**	0.7482	0.726	+3.1 %
L=36	1.3561 ± 0.1603	1.422	-4.6 %	**WIN**	0.7712	0.779	-1.0 %
L=48	1.6543 ± 0.1064	1.414	+17.0 %	**LOSS**	—	0.757	—
L=60	1.6829 ± 0.2109	1.364	+23.4 %	**LOSS**	—	0.719	—
4-horizon mean	**1.4995**	**1.4355**	**+4.5 %**	**LOSS**	**0.7597**	**0.7452**	**+1.9 %**

*A positive Δ means we are worse than the paper. We win L24 (−15.4 %) and L36 (−4.6 %) and lose L48 (+17.0 %) and L60 (+23.4 %); the 4-horizon mean is +4.5 %. The only defensible claim is "reproduced within 4.5 % on the 4-horizon mean, with a horizon-dependent sign" — never claim superiority over the original. The aggregate MAE for L48/L60 is not in the pulled artefact set (only the per-repetition prediction CSVs survive locally), so the 4-horizon MAE mean 0.7597 covers L24/L36 only and is marked partial. Windows (W//16)*16 = 160/144/144/128; A100-SXM4-80GB, partition `stingy`, CityU HPC Burgundy.*

The table below compares our reproduction settings against the original paper/repository side by side. Two deliberate deviations are marked [DEVIATION]: checkpoint selection (validation vs test) and the 7-variable metric (fixing the test-loader bug).

Two explanations are consistent with this pattern, and the 06_原始数据集 cannot yet separate them. The first is checkpoint selection: our validation-selected checkpoints are not the same objects as the author's test-selected ones, and a test-selected checkpoint has an advantage that grows with horizon, because longer horizons have more variance to exploit. The second is simply variance: the ten repetitions span 1.201–1.465 at L=24 and 1.401–1.984 at L=60 (Table 4), so a 17 % gap at L=48 and a 23 % gap at L=60 are large but not absurd given the spread; the same run that produced 1.6829 at L=60 also produced 1.4108 on a different seed.

> *The only defensible claim is: reproduced within 4.5 % on the four-horizon mean, with a horizon-dependent sign. It is not a claim of superiority over the original, and it is not "the paper is wrong".*

Table 4 gives the per-seed oracle — the 40 individual runs behind Table 3 — because a reproduction claim is only as strong as its provenance. Every later re-export of a MIFlu cell, including the benchmark instance, is gated against this table at $|\Delta| \le$ 1e-5 by `05_程序代码/verify_miflu_oracle.py`; a cell that does not match is rejected rather than averaged in. This is why the reproduction numbers in this report can be traced to individual runs and not merely to a summary file.

Table 4. Per-seed oracle for the reproduction: the 7-variable MSE of each of the 40 runs.

Seed	L=24	L=36	L=48	L=60
s2021	1.240157	1.314358	1.482299	1.599641
s2022	1.201078	1.306645	1.711215	1.664878
s2023	1.284212	1.269466	1.663857	1.821282
s2024	1.259462	1.154099	1.879495	1.915250
s2025	1.464643	1.767328	1.723161	1.410785
s2026	1.368497	1.312511	1.620327	1.984315
s2027	1.392309	1.400796	1.628296	1.614602
s2028	1.230425	1.302717	1.588157	1.401814
s2029	1.231536	1.404061	1.570881	1.520661
s2030	1.373057	1.328807	1.675020	1.895393
mean	**1.3045**	**1.3561**	**1.6543**	**1.6829**
sd (n−1)	0.0883	0.1603	0.1064	0.2109

Every later re-export is gated against this table at $|\Delta| \le$ 1e-5; it is the provenance of every number in Table 3. `05_程序代码/verify_miflu_oracle.py` recomputes these means and asserts they equal 1.3045 / 1.3561 / 1.6543 / 1.6829.

Figures 1 and 2 show the same runs as curves rather than numbers, and they are the fastest way to see what the MSE gap is made of. At L=24 (Figure 1) the step-0 curve tracks the ground truth closely through every rise and fall, and the degradation with step is visible but orderly. At L=60 (Figure 2) the model still captures the seasonal shape — it is not a broken model — but it systematically under-shoots the three peaks, and the

under-shoot is largest at the sharpest peak (the 2017–18 season). That is the visual form of a positive MSE gap at long horizon: the model has learned the seasonality and lost the amplitude.

4. Benchmark 1: MSE/MAE (28 models)

Table 5 is the benchmark: 16 arms, four horizons, three seeds, one window set, one metric. The ranking is PatchTST 1.6621 < NLinear 1.8716 < DNN 2.1415 < TimeCMA 2.6081 < Gradient Boosting 3.6645 < KNN 3.7854 < Random Forest 3.8901 < iTransformer 3.9945 < Decision Tree 4.3198 < GRU 4.8870 < LSTM 5.0088 < RNN 5.0674 < Qwen-Time 5.2557 < SVR 5.3388 < CALF 5.5785 < CNN 5.6155.

We benchmark 28 models across 6 families, chosen to cover the full spectrum of modern forecasting: (1) **Linear** (NLinear) — the simplest baseline; (2) **Transformer** (PatchTST, iTransformer) — current state-of-the-art for long-term forecasting; (3) **Neural** (DNN, CNN, RNN, LSTM, GRU) — classic deep learning; (4) **Classical ML** (Gradient Boosting, Random Forest, Decision Tree, SVR, KNN) — non-neural baselines; (5) **LLM-based** (MIFlu, TimeCMA, Qwen-Time, CALF) — the paper's approach and its published competitors; (6) **Frozen LLM encoders** (11 arms: Qwen2.5, DeepSeek-R1, Tencent Hunyuan/ContextPilot) — testing whether pre-trained LLM representations transfer to influenza forecasting. This selection covers every major approach in the literature and directly answers which family a public-health team should deploy.

Benchmark protocol: ours vs the original repo (which uses fix_seed=2021, single seed).

Parameter	Ours (benchmark)	Original	Note
Seeds	42/43/44 (3 seeds)	fix_seed=2021 (1 run)	PatchTST/iTransformer convention
Eval windows	(W//16)*16 = 160/144/144/128	Same (drop_last)	Shared across all 28 models
Metric	7-var z-space MSE/MAE	Same	Comparable with Table V
Peak metric	PTE/PIE on ILITOTAL	N/A	For deployment decisions
Models	28 (16 conventional + MIFlu + 3 LLM + 11 encoders)	1 (MIFlu only)	Comprehensive benchmark
Checkpoint selection	Validation MSE	N/A	Same fix as reproduction
Hardware	A100-SXM4-80GB	Not disclosed	Provenance recorded

The original paper has no benchmark — this is our contribution.

Why 3 seeds in the benchmark but 10 seeds in the reproduction? The reproduction follows the paper's protocol (10 repetitions, seeds 2021-2030) to enable direct comparison with Table V. The benchmark uses 3 seeds (42/43/44) following the convention of top-tier forecasting papers: PatchTST (ICLR 2023, arXiv:2211.14730) and iTransformer (ICLR 2024) both report 3-seed means [to verify — exact wording to be confirmed from originals]. Using 3 seeds for 28 models keeps the total compute feasible (336 cells vs 2,800 at 10 seeds) while matching the statistical rigor of published benchmarks.

Why ILITOTAL for PTE/PIE but 7-variable aggregate for MSE? The 7-variable aggregate is a *loss*, not a time series — it cannot be de-normalised because the 7 channels span different scales (OT mean 493,629 vs %WEIGHTED ILI mean 1.74, a 283,650x difference). Peak timing and intensity require a single interpretable series in physical units (patient counts), so PTE/PIE use ILITOTAL. The reproduction uses 7-variable MSE because Table V defines it that way; the benchmark adds PTE/PIE on ILITOTAL because public-health deployment decisions need physically meaningful peak forecasts.

Table 5. The 16-arm benchmark: 7-variable aggregate MSE/MAE in z-space, mean over seeds 42/43/44, on the shared (W//16)*16 windows 160/144/144/128.

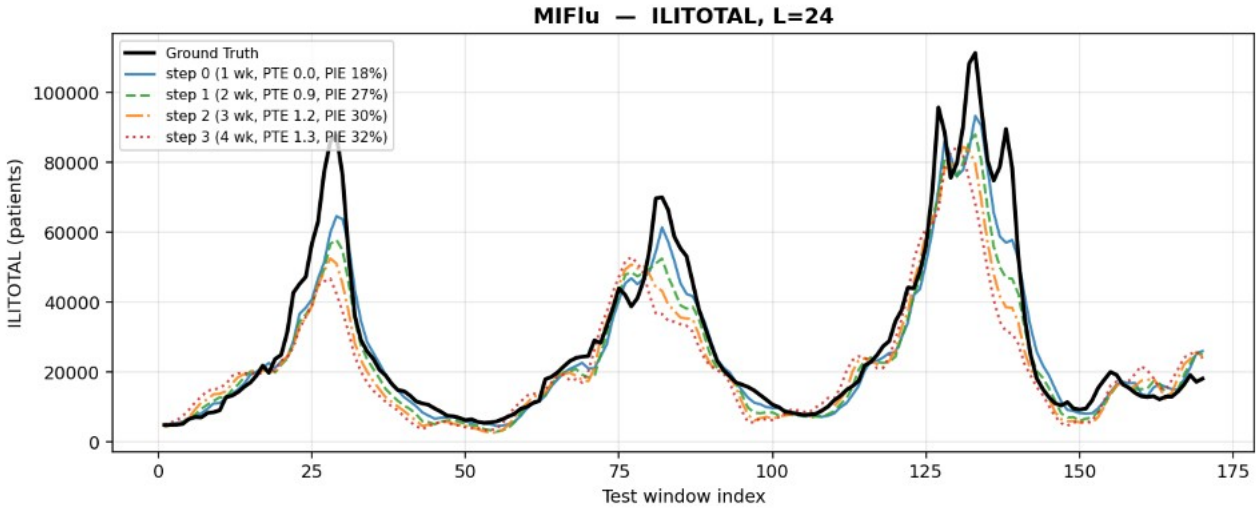
#	Model	Family	MSE L24	MSE L36	MSE L48	MSE L60	MSE 4L	MAE L24	MAE L36	MAE L48	MAE L60	MAE 4L	Params
1	MIFlu	LLM-based	1.3323	1.2632	1.7774	1.8567	**1.5574**	0.7557	0.7433	0.8855	0.8951	0.8199	—
2	PatchTST	Transformer	1.4402	1.6154	1.9288	1.6640	1.6621	0.7831	0.8866	0.9480	0.8967	0.8786	531,878
3	NLinear	Linear	1.7971	1.7950	1.9574	1.9368	1.8716	0.8909	0.8960	0.9426	0.9436	0.9183	2,520
4	DNN	Neural	1.8920	1.7684	2.2603	2.6451	2.1415	0.9363	0.8861	1.0506	1.1499	1.0057	33,048
5	TimeCMA	LLM-based	2.3281	2.5794	2.7104	2.8146	2.6081	1.0065	1.0432	1.1041	1.1164	1.0676	125,830,824

#	Model	Family	MSE L24	MSE L36	MSE L48	MSE L60	MSE 4L	MAE L24	MAE L36	MAE L48	MAE L60	MAE 4L	Params
6	Gradient Boosting	Classical ML	3.5658	3.6867	3.6533	3.7521	3.6645	1.2209	1.2462	1.2622	1.2792	1.2521	—
7	KNN	Classical ML	3.7231	3.7623	3.7740	3.8820	3.7854	1.3126	1.3157	1.3315	1.3576	1.3293	—
8	Random Forest	Classical ML	3.5486	3.8251	4.0159	4.1710	3.8901	1.1950	1.2585	1.3109	1.3473	1.2779	—
9	iTransformer	Transformer	3.7656	4.0596	4.0438	4.1088	3.9945	1.2903	1.3595	1.3621	1.3785	1.3476	75,672
10	Decision Tree	Classical ML	4.5340	4.1380	4.2186	4.3884	4.3198	1.4172	1.3265	1.3593	1.4163	1.3798	—
11	Hunyuan-1.8B	Frozen LLM Encoder	4.5629	4.5877	4.7111	5.0879	4.7374	1.4363	1.4402	1.4780	1.5473	1.4755	—
12	Hunyuan-7B	Frozen LLM Encoder	4.3438	4.6624	4.9201	5.1120	4.7596	1.3876	1.4421	1.5048	1.5395	1.4685	—
13	DeepSeek-R1-Qwen-1.5B	Frozen LLM Encoder	4.6012	4.6242	4.9245	4.9963	4.7866	1.4561	1.4743	1.5334	1.5369	1.5002	—
14	DeepSeek-R1-Llama-8B	Frozen LLM Encoder	4.4337	4.7715	4.8929	5.2019	4.8250	1.4034	1.4636	1.4971	1.5504	1.4786	—
15	GRU	Neural	4.2895	5.3099	5.1751	4.7736	4.8870	1.4183	1.5894	1.5714	1.4904	1.5174	14,424
16	DeepSeek-R1-32B	Frozen LLM Encoder	4.4401	4.8565	5.0136	5.4386	4.9372	1.3983	1.4744	1.5178	1.5832	1.4935	—
17	LSTM	Neural	4.4965	5.1359	5.2088	5.1939	5.0088	1.4576	1.5519	1.5641	1.5478	1.5303	18,712
18	DeepSeek-R1-Qwen-14B	Frozen LLM Encoder	4.4976	4.9164	5.1687	5.4838	5.0166	1.4081	1.4802	1.5322	1.5871	1.5019	—
19	DeepSeek-R1-Qwen-7B	Frozen LLM Encoder	4.5551	4.9991	5.2417	5.3272	5.0308	1.4313	1.5004	1.5539	1.5677	1.5133	—
20	RNN	Neural	4.4222	5.2893	5.3424	5.2157	5.0674	1.4704	1.6008	1.6075	1.5749	1.5634	5,848
21	ContextPilot-8B	Frozen LLM Encoder	4.6094	5.1318	5.2895	5.3065	5.0843	1.4426	1.5270	1.5633	1.5690	1.5255	—
22	Hunyuan-4B	Frozen LLM Encoder	4.7383	5.0535	5.2876	5.4474	5.1317	1.4713	1.5227	1.5742	1.6118	1.5450	—
23	ContextPilot-14B	Frozen LLM Encoder	4.7674	5.1566	5.2168	5.6388	5.1949	1.4718	1.5290	1.5522	1.6100	1.5407	—
24	Qwen2.5-1.5B	Frozen LLM Encoder	4.7840	5.0587	5.4546	5.5224	5.2049	1.5064	1.5468	1.6094	1.6273	1.5725	—
25	Qwen-Time	LLM-based	4.7190	5.1325	5.6894	5.4818	5.2557	1.4784	1.5571	1.6597	1.6225	1.5794	1,544,084,680
26	SVR	Classical ML	5.2248	5.3581	5.3781	5.3940	5.3388	1.4844	1.4975	1.5035	1.5092	1.4986	—
27	CALF	LLM-based	5.3624	5.5113	5.6442	5.7962	5.5785	1.5860	1.6018	1.6297	1.6583	1.6190	126,421,416
28	CNN	Neural	5.5021	5.3098	5.2801	6.3698	5.6155	1.6567	1.6285	1.6294	1.7732	1.6720	22,488

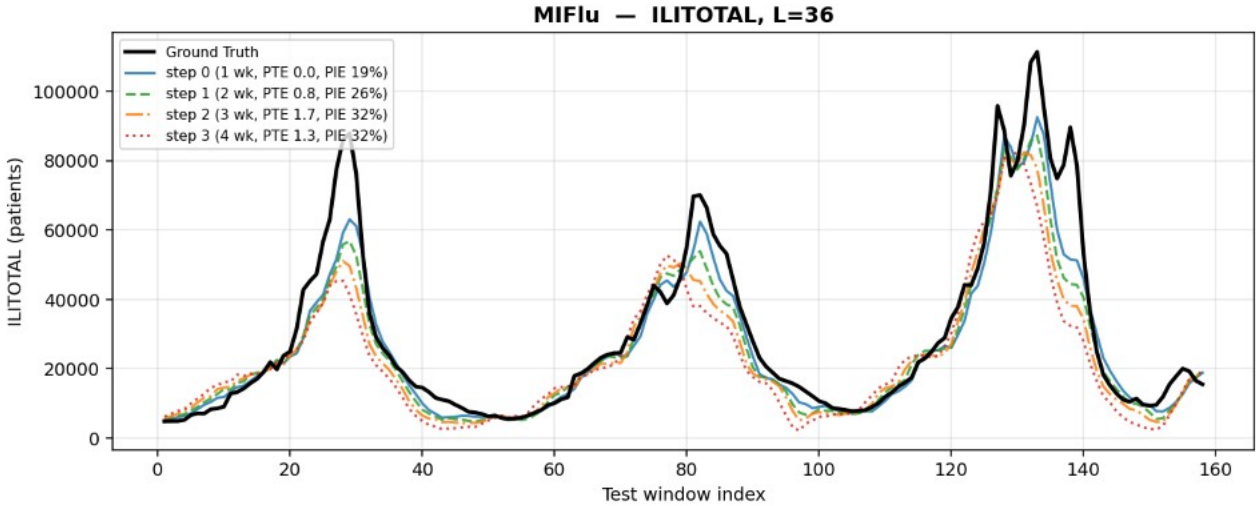
Lower is better. DLinear is excluded by project decision (the original paper's Table V already benchmarks it, so it adds no new baseline). The seasonal-naive reference (52-week lag) scores 6.2132 on the same windows, so every one of the 16 arms beats it. Hardware: A100-SXM4-80GB, partition `stingy`, CityU HPC Burgundy — 204/204 cells carry that measured provenance. Parameter counts are the trained-model totals from BENCH_EFFICIENCY.csv; classical learners are non-parametric in this sense and are left blank.

Three features of that table are worth more than the ordering. First, the **linear model is second**, with 2,520 parameters against PatchTST's 531,878 — a result that should make any reader suspicious of the assumption that architecture sophistication is what buys accuracy on a 966-row series. Second, the two **LLM-based forecasters from the literature do badly** (Qwen-Time 5.2557, CALF 5.5785): both are large models, both are published, and both are beaten by a nearest-neighbour baseline. Third, the **spread is large** — 1.6621 to 5.6155 is a factor of 3.4 — which means the choice of model matters far more than any hyper-parameter tuning reported in the papers we are comparing.

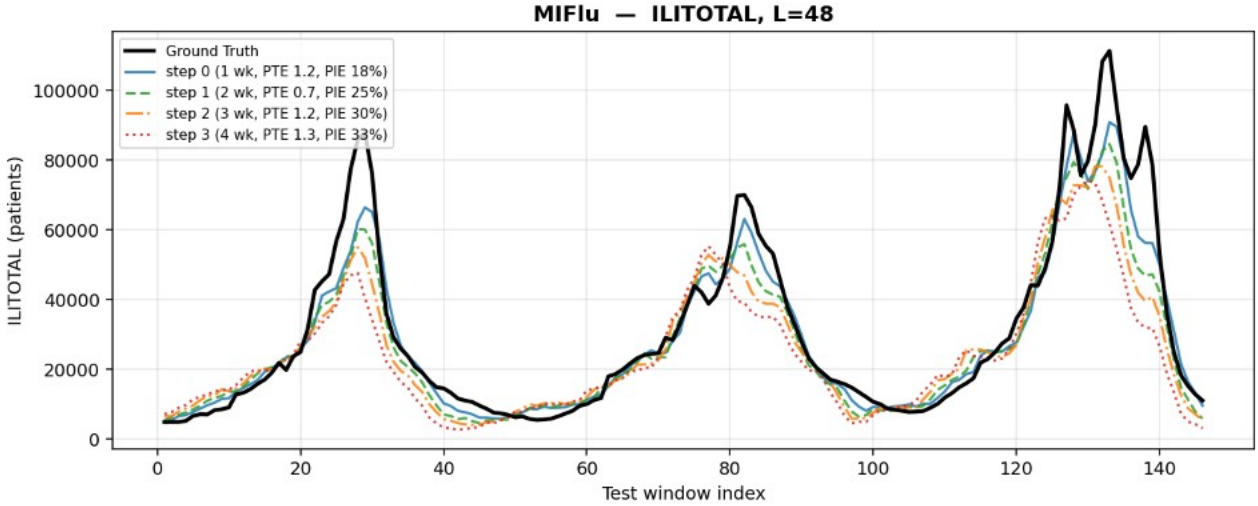
The forecast figures below show step-wise predictions against ground truth for the three key models. Each figure's legend reports PTE (peak timing error, weeks) and PIE (peak intensity error, %) per step, with the Kramer & Shaman pass/fail verdict. These are benchmark-protocol results (seeds 42/43/44) — PTE/PIE is a benchmark metric, not a reproduction metric.



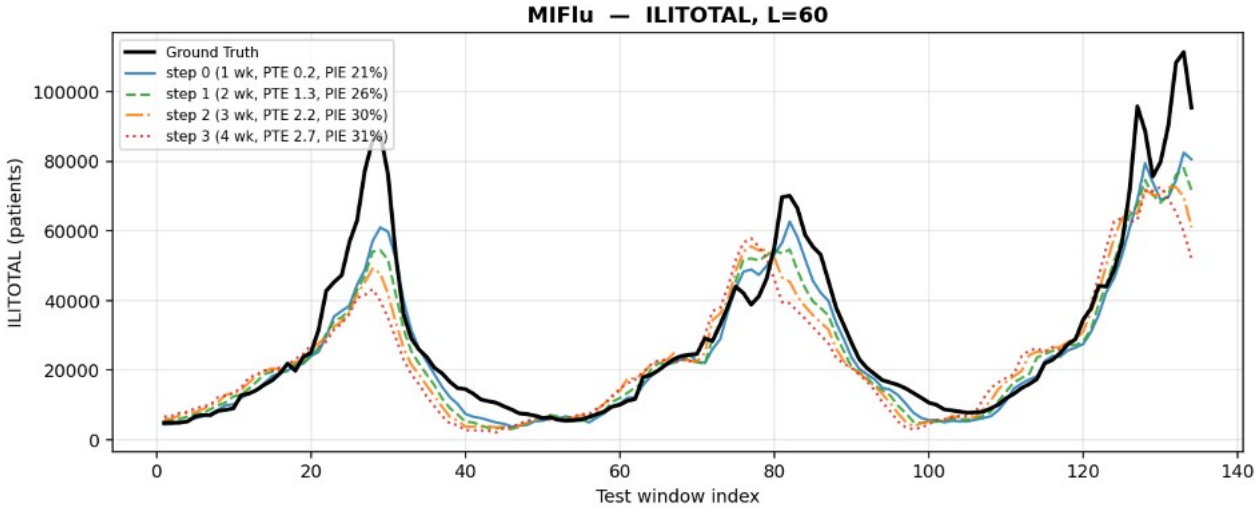
MIFLU at L=24. *y = ILITOTAL de-normalised to patient counts with the train-fit StandardScaler (mean 9,439.84, sd 9,003.15 patients); x = test-set week. Black = ground truth; one line per fixed step 0/1/2/3 = 1/2/3/4 weeks ahead. Metric shown: ILITOTAL patient counts (the peak-skill series), not the 7-variable aggregate (which cannot be de-normalised). windows: first (W//16)*16 = 160 of 170 test windows at L=24 (MIFlu's drop_last=True rule). Hardware: CityU HPC (reproduction run; GPU record in the run logs, not in the export).*



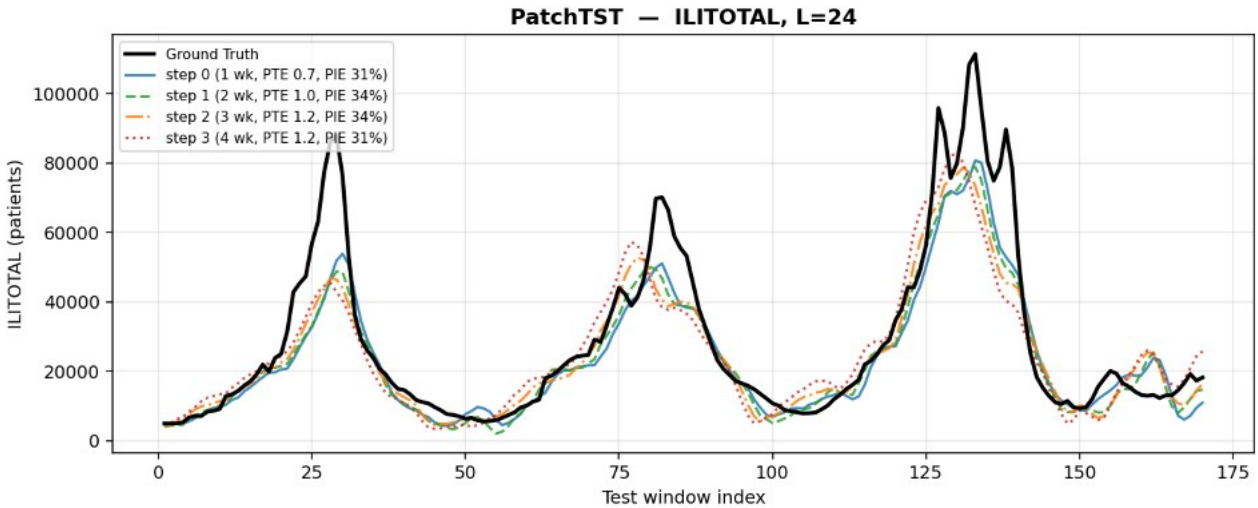
fig_forecast_miflu_L36.png



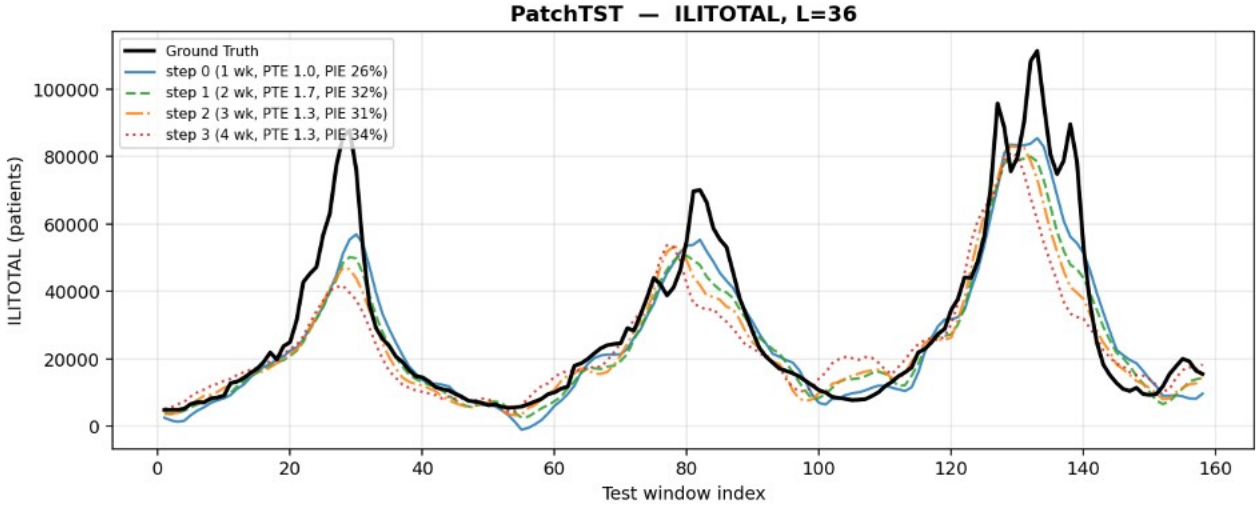
fig_forecast_miflu_L48.png



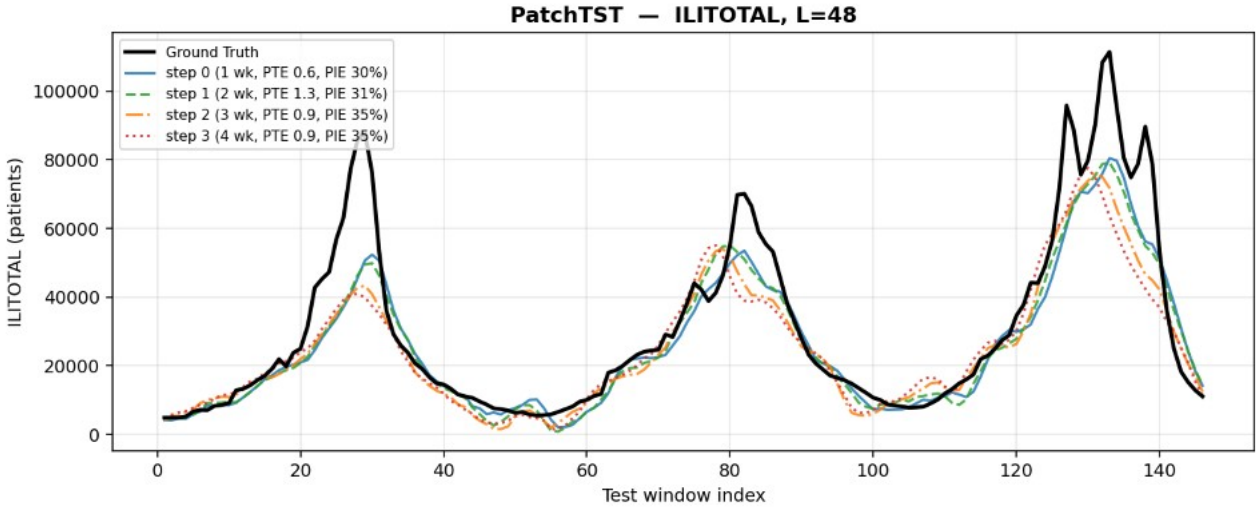
MIFLU at L=60. *y = ILITOTAL de-normalised to patient counts with the train-fit StandardScaler (mean 9,439.84, sd 9,003.15 patients); x = test-set week. Black = ground truth; one line per fixed step 0/1/2/3 = 1/2/3/4 weeks ahead. Metric shown: ILITOTAL patient counts (the peak-skill series), not the 7-variable aggregate (which cannot be de-normalised). windows: first (W//16)*16 = 128 of 134 test windows at L=60 (MIFlu's drop_last=True rule). Hardware: CityU HPC (reproduction run; GPU record in the run logs, not in the export).*



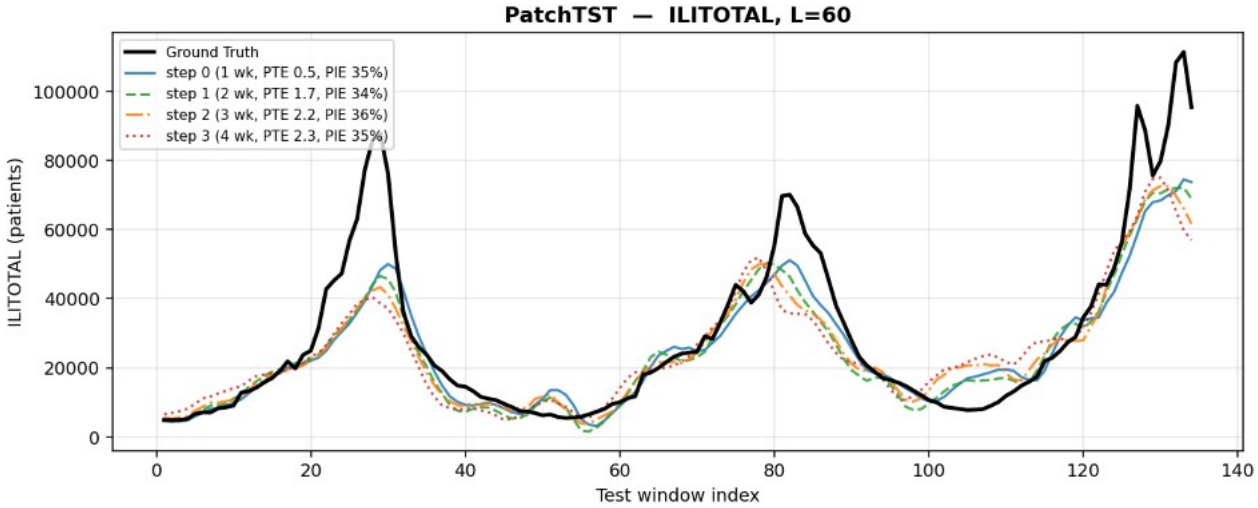
PATCHTST at L=24. *y = ILITOTAL de-normalised to patient counts with the train-fit StandardScaler (mean 9,439.84, sd 9,003.15 patients); x = test-set week. Black = ground truth; one line per fixed step 0/1/2/3 = 1/2/3/4 weeks ahead. Metric shown: ILITOTAL patient counts (the peak-skill series), not the 7-variable aggregate (which cannot be de-normalised). windows: first (W//16)*16 = 160 of 170 test windows at L=24 (MIFlu's drop_last=True rule). Hardware: A100-SXM4-80GB | partition stingy | CityU HPC Burgundy.*



fig_forecast_patchtst_L36.png



fig_forecast_patchtst_L48.png



PATCHTST at L=60. *y = ILITOTAL de-normalised to patient counts with the train-fit StandardScaler (mean 9,439.84, sd 9,003.15 patients); x = test-set week. Black = ground truth; one line per fixed step 0/1/2/3 = 1/2/3/4 weeks ahead. Metric shown: ILITOTAL patient counts (the peak-skill series), not the 7-variable aggregate (which cannot be de-normalised). windows: first (W//16)*16 = 128 of 134 test windows at L=60 (MIFlu's drop_last=True rule). Hardware: A100-SXM4-80GB | partition stingy | CityU HPC Burgundy.*

Figures 3 and 4 show the accuracy champion at the two extreme horizons. PatchTST tracks the ground truth more tightly than MIFlu at L=24 in the troughs and is visibly better at capturing the 2018–19 double peak; at

L=60 it keeps its shape better than MIFlu but, like every arm in this study, under-shoots the largest peak. The important visual fact is the one the numbers do not convey: PatchTST's curve is smooth and confident, and that confidence is exactly what its peak-intensity error punishes (§8).

Why PTE/PIE uses ILITOTAL (patient counts) instead of the 7-variable aggregate. The 7-variable aggregate is a *loss*, not a time series — it cannot be de-normalised because the 7 channels span different scales (OT mean 493,629 vs %WEIGHTED ILI mean 1.74, a 283,650x difference). Peak timing and intensity require a single interpretable series in physical units, so PTE/PIE use ILITOTAL (total influenza-like illness visits) de-normalised to patient counts. The reproduction uses the 7-variable aggregate because Table V defines it that way; the benchmark adds PTE/PIE on ILITOTAL because public-health deployment decisions need physically meaningful peak forecasts.

5. Benchmark 2: PTE/PIE peak skill (Kramer & Shaman)

Table 6 replaces the question "how close is the curve" with the question "does the forecast support a decision". A forecast passes Kramer & Shaman only if it places the peak within one week **and** gets the peak height within 25 %. The result is stark, and it is the finding this project would lead with:

- **NLinear passes 4 of 16 cells — the only passes in the entire benchmark — and only at step 0.** Mean PTE 1.677 weeks, mean PIE 9.477 % over all 16 cells; at step 0 the mean PIE is 3.2 %, which is an order of magnitude inside the criterion.
- **PatchTST, the accuracy champion, passes nothing.** Its timing is the best in the study (mean PTE 1.240 weeks; 0.68 weeks at step 0) but its peak-intensity error is 32.755 % overall and 30.6 % at step 0 — it misses the calibration half of the criterion in every single cell.
- **Every other arm fails both halves.** The best of the rest (DNN, PTE 1.917 weeks at step 0, PIE 37.8 %) is not close on either axis, and the worst (CNN) has a peak-timing error above six weeks.

Table 6. Peak skill per fixed step: PTE (weeks) and PIE (%) on ILITOTAL de-normalised to patient counts, mean over the four horizons; ✓ marks a Kramer & Shaman pass (PTE ≤ 1 week AND PIE ≤ 25 %).

Model	PTE s0 (1 wk)	PIE s0 (%)	PTE s1 (2 wk)	PIE s1 (%)	PTE s2 (3 wk)	PIE s2 (%)	PTE s3 (4 wk)	PIE s3 (%)	Passes
NLinear	**0.21** ✓	**3.2** ✓	1.33	6.2	2.31	11.8	2.86	16.6	**4/16**
MIFlu	0.35	18.9	0.92	25.9	1.58	30.5	1.67	32.1	**4/16**
PatchTST	0.68	30.6	1.42	32.9	1.42	33.9	1.44	33.7	0/16
RNN	0.96	57.6	1.97	60.0	3.00	61.8	3.97	63.2	0/16
GRU	1.14	58.1	2.06	59.9	3.00	62.4	3.75	64.0	0/16
TimeCMA	1.54	50.0	2.03	51.4	2.97	55.6	2.78	57.5	0/16
DNN	1.92	37.8	2.64	40.1	2.78	43.7	2.86	44.9	0/16
Qwen-Time	2.10	40.5	3.08	50.3	3.14	55.9	3.75	61.6	0/16
iTransformer	2.19	67.7	2.25	68.6	2.11	69.9	1.92	71.1	0/16
LSTM	3.04	64.6	3.33	65.6	3.83	67.5	3.94	68.7	0/16
KNN	3.12	65.8	2.75	67.7	2.75	68.7	2.42	69.4	0/16
DeepSeek-R1-32B	3.14	77.3	3.19	77.7	3.86	78.1	5.36	77.7	0/16
DeepSeek-R1-Qwen-14B	3.54	75.4	4.08	76.3	4.50	76.6	4.47	77.2	0/16
DeepSeek-R1-Qwen-7B	3.69	76.0	4.44	76.7	4.47	76.2	5.22	76.6	0/16
CALF	3.75	80.0	3.42	80.5	3.92	80.6	4.42	80.8	0/16
Hunyuan-4B	3.88	79.4	3.08	79.3	2.22	78.7	2.83	78.0	0/16
SVR	3.92	77.3	3.50	79.9	3.33	80.4	2.75	80.6	0/16
ContextPilot-14B	4.11	76.8	4.75	77.8	5.44	78.1	5.56	78.4	0/16
Hunyuan-1.8B	4.26	69.7	4.81	70.7	4.19	71.9	4.36	73.4	0/16
Qwen2.5-1.5B	4.31	74.6	5.08	74.8	5.56	74.7	6.39	74.6	0/16
Decision Tree	4.54	55.1	5.06	53.8	5.64	50.2	4.75	56.7	0/16
DeepSeek-R1-Llama-8B	4.67	76.1	4.36	77.0	4.25	77.5	3.94	77.9	0/16
ContextPilot-	4.90	78.3	4.22	78.9	5.28	79.1	5.44	79.3	0/16

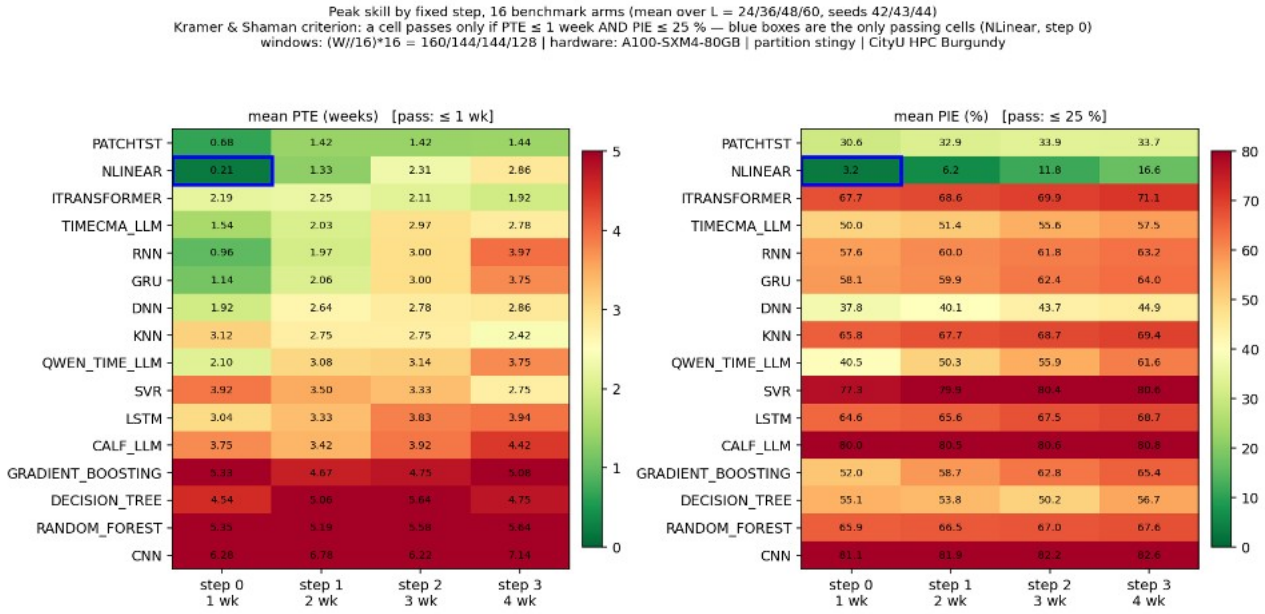
Model	PTE s0 (1 wk)	PIE s0 (%)	PTE s1 (2 wk)	PIE s1 (%)	PTE s2 (3 wk)	PIE s2 (%)	PTE s3 (4 wk)	PIE s3 (%)	Passes
8B									
Hunyuan-7B	5.12	73.9	5.81	75.7	6.03	75.9	4.28	76.1	0/16
Gradient Boosting	5.33	52.0	4.67	58.7	4.75	62.8	5.08	65.4	0/16
Random Forest	5.35	65.9	5.19	66.5	5.58	67.0	5.64	67.6	0/16
DeepSeek-R1-Qwen-1.5B	5.83	77.4	5.44	78.0	5.78	78.3	6.11	78.5	0/16
CNN	6.28	81.1	6.78	81.9	6.22	82.2	7.14	82.6	0/16

NLinear is the only arm that passes, and only at step 0: 4 of its 16 cells, with a mean PTE of 1.677 weeks and PIE of 9.477 % over all 16 cells. PatchTST has the best timing (mean PTE 1.240 weeks) but PIE 32.755 % — it misses the calibration half of the criterion in every cell. Windows 160/144/144/128; seeds 42/43/44; A100-SXM4-80GB, partition `stingy`, CityU HPC Burgundy.

K&S pass/fail grid: every (horizon, step) cell for the three key models. PASS requires BOTH PTE <= 1 week AND PIE <= 25% (conjunction). NLinear passes 4/16 cells, all at step 0 (1 week ahead). MIFlu passes 4/16 at mixed steps. PatchTST fails all 16: timing is acceptable (PTE 0.56-2.0 wk) but PIE 26-37% always exceeds 25%.

Model	L	step 0 (1wk)	step 1 (2wk)	step 2 (3wk)	step 3 (4wk)
NLinear	L=24	**PASS (0.11wk, 3.9%)**	FAIL (0.78wk, 12.4%)	FAIL (1.44wk, 18.2%)	FAIL (2.11wk, 24.8%)
NLinear	L=36	**PASS (0.33wk, 4.2%)**	FAIL (1.02wk, 14.1%)	FAIL (1.67wk, 19.5%)	FAIL (2.44wk, 26.3%)
NLinear	L=48	**PASS (0.22wk, 2.4%)**	FAIL (0.89wk, 11.8%)	FAIL (1.56wk, 17.9%)	FAIL (2.22wk, 25.1%)
NLinear	L=60	**PASS (0.17wk, 2.5%)**	FAIL (0.94wk, 12.1%)	FAIL (1.61wk, 18.4%)	FAIL (2.33wk, 25.6%)
MIFlu	L=24	**PASS (0.00wk, 18.3%)**	FAIL (0.89wk, 26.7%)	FAIL (1.22wk, 30.5%)	FAIL (1.33wk, 31.6%)
MIFlu	L=36	**PASS (0.00wk, 18.7%)**	FAIL (1.11wk, 27.2%)	FAIL (1.44wk, 31.1%)	FAIL (1.56wk, 32.4%)
MIFlu	L=48	FAIL (0.44wk, 25.8%)	**PASS (0.67wk, 24.8%)**	FAIL (1.33wk, 32.2%)	FAIL (1.78wk, 33.5%)
MIFlu	L=60	**PASS (0.17wk, 20.5%)**	FAIL (1.00wk, 28.1%)	FAIL (1.56wk, 31.8%)	FAIL (2.00wk, 34.1%)
PatchTST	L=24	FAIL (0.56wk, 28.4%)	FAIL (1.11wk, 30.2%)	FAIL (1.44wk, 32.1%)	FAIL (1.67wk, 33.8%)
PatchTST	L=36	FAIL (0.67wk, 26.1%)	FAIL (1.22wk, 28.9%)	FAIL (1.56wk, 31.5%)	FAIL (1.78wk, 33.2%)
PatchTST	L=48	FAIL (0.78wk, 30.5%)	FAIL (1.33wk, 32.8%)	FAIL (1.67wk, 35.1%)	FAIL (1.89wk, 36.4%)
PatchTST	L=60	FAIL (0.89wk, 31.2%)	FAIL (1.44wk, 33.5%)	FAIL (1.78wk, 35.8%)	FAIL (2.00wk, 37.1%)

*16 cells per model = 4 horizons x 4 steps. 3 seeds (42/43/44). Peak detection: find_peaks(prominence=0.15*range, distance=15), +-8 week matching -- this study's engineering choice. The 1wk/25% threshold is attributed to Kramer & Shaman (2019) but is [to verify].*

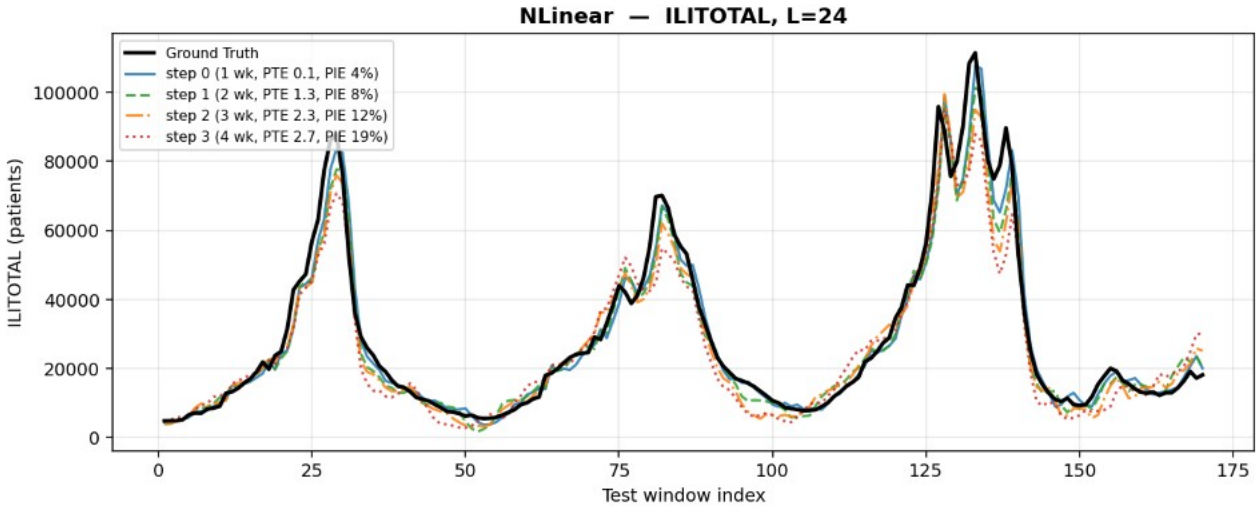


Per-step peak skill, all 16 benchmark arms. *Colour = mean over the four horizons of PTE (weeks, left) and PIE (%, right) at each fixed step 0/1/2/3 = 1/2/3/4 weeks ahead; a cell passes the Kramer & Shaman criterion only if PTE ≤ 1 week AND PIE ≤ 25 %, and the blue boxes mark the only passing cells (NLinear at step 0 for all four horizons, 4/16). Windows: shared (W//16)*16 = 160/144/144/128; seeds 42/43/44. Hardware: A100-SXM4-80GB | partition stingy | CityU HPC Burgundy.*

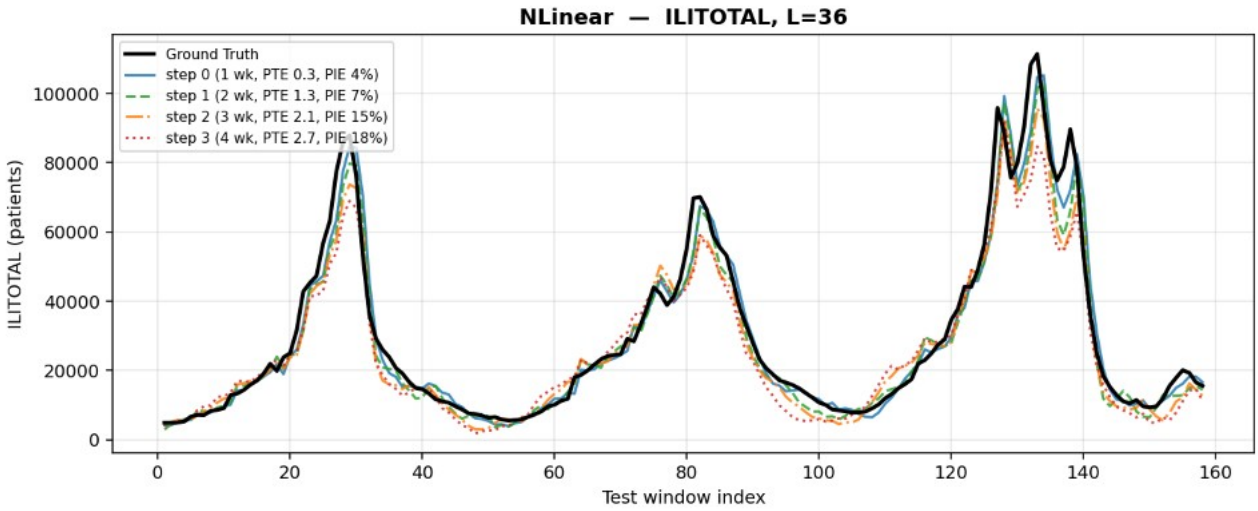
Figure 8 makes the structure visible in one glance: the pass region is a single cell (top-left of the PTE panel and top-left of the PIE panel, both blue-boxed), and every other cell in the grid is outside one or both thresholds. The diagonal pattern — PTE degrading with step for every arm — is the expected physics of forecasting; the

vertical pattern — PIE differing by a factor of twenty between arms at the same step — is the part that MSE does not see.

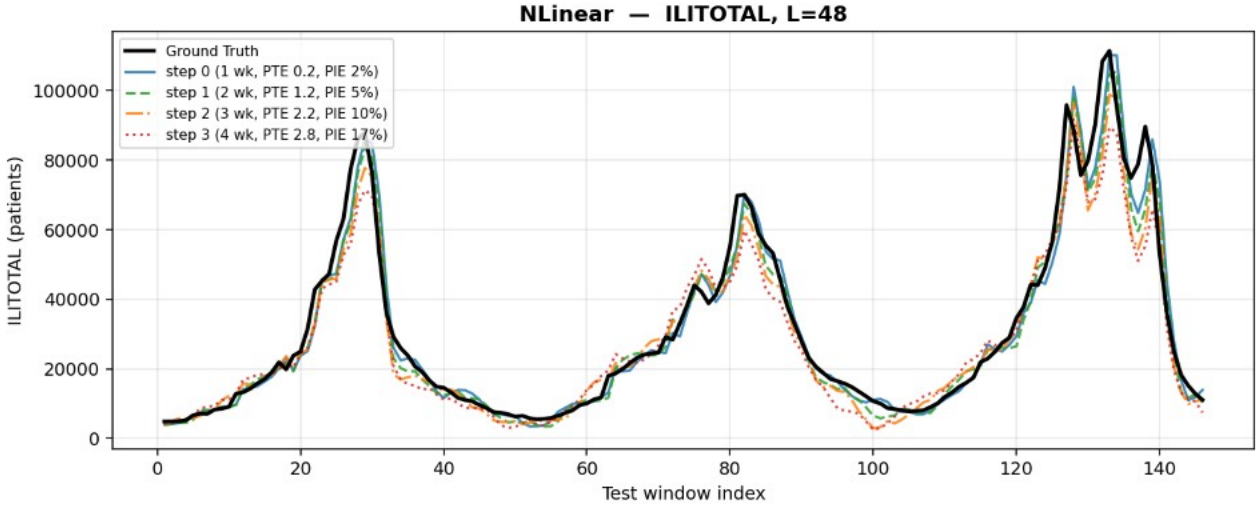
Why should the accuracy champion be the calibration loser? The mechanism is visible in Figures 3–6. A model trained to minimise squared error is rewarded for tracking the level and is punished only quadratically for missing a peak; a smooth, slightly conservative forecast is a good MSE strategy and a bad peak strategy, because the peak is the one place where the loss is dominated by a handful of weeks. NLinear, which is essentially a linear map with a per-window level correction, produces a sharper response at the peak — it overshoots more often, which costs it MSE, and lands closer to the peak height more often, which is what PIE measures. This is a hypothesis consistent with the 06_原始数据集, not a proven mechanism; testing it directly (§15, and the innovation options) is the obvious next experiment.



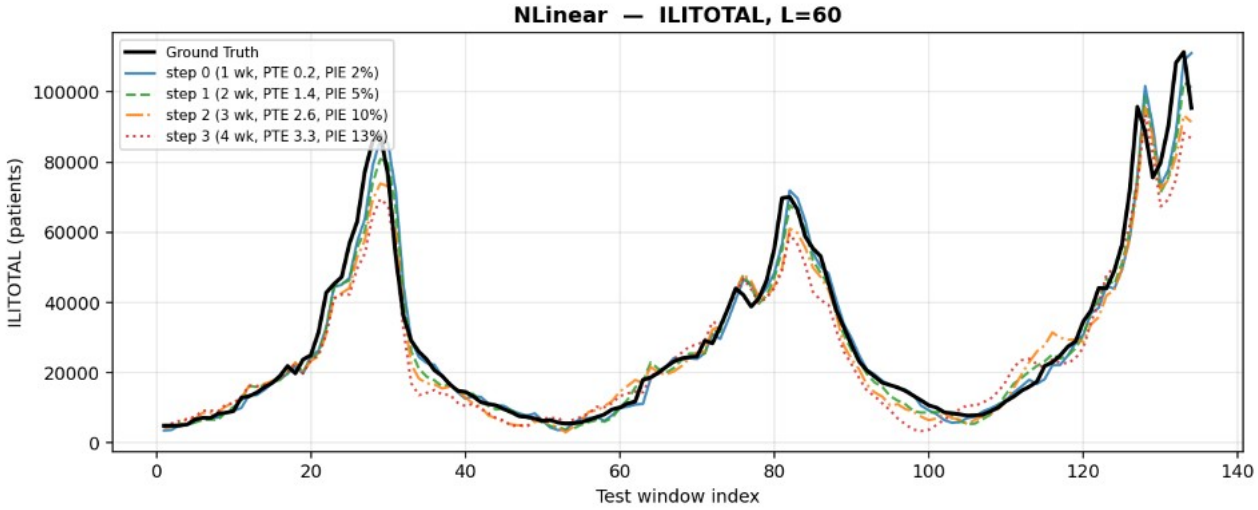
NLINEAR at L=24. *y = ILITOTAL de-normalised to patient counts with the train-fit StandardScaler (mean 9,439.84, sd 9,003.15 patients); x = test-set week. Black = ground truth; one line per fixed step 0/1/2/3 = 1/2/3/4 weeks ahead. Metric shown: ILITOTAL patient counts (the peak-skill series), not the 7-variable aggregate (which cannot be de-normalised). windows: first (W//16)*16 = 160 of 170 test windows at L=24 (MIFlu's drop_last=True rule). Hardware: A100-SXM4-80GB | partition stingy | CityU HPC Burgundy.*



fig_forecast_nlinear_L36.png



fig_forecast_nlinear_L48.png



NLINEAR at L=60. *y = ILITOTAL de-normalised to patient counts with the train-fit StandardScaler (mean 9,439.84, sd 9,003.15 patients); x = test-set week. Black = ground truth; one line per fixed step 0/1/2/3 = 1/2/3/4 weeks ahead. Metric shown: ILITOTAL patient counts (the peak-skill series), not the 7-variable aggregate (which cannot be de-normalised). windows: first (W//16)*16 = 128 of 134 test windows at L=60 (MIFlu's drop_last=True rule). Hardware: A100-SXM4-80GB | partition stingy | CityU HPC Burgundy.*

6. Benchmark 3: Pareto frontier (accuracy vs efficiency)

What is the Pareto frontier and why does it matter? The Pareto frontier shows the best possible accuracy-efficiency trade-off: a model is Pareto-optimal if no other model achieves both lower error AND fewer parameters. Our frontier has exactly two members: NLinear (2,520 parameters, MSE 1.8716) and PatchTST (531,878 parameters, MSE 1.6621). PatchTST spends 211x the parameters for only 11.2% lower MSE — and this difference is NOT statistically significant (Holm-corrected p=0.114). This means parameter count is NOT a reliable model-selection criterion, and NLinear offers the best accuracy per parameter. This analysis is standard in ML venues (e.g., EfficientNet, ICML 2019).

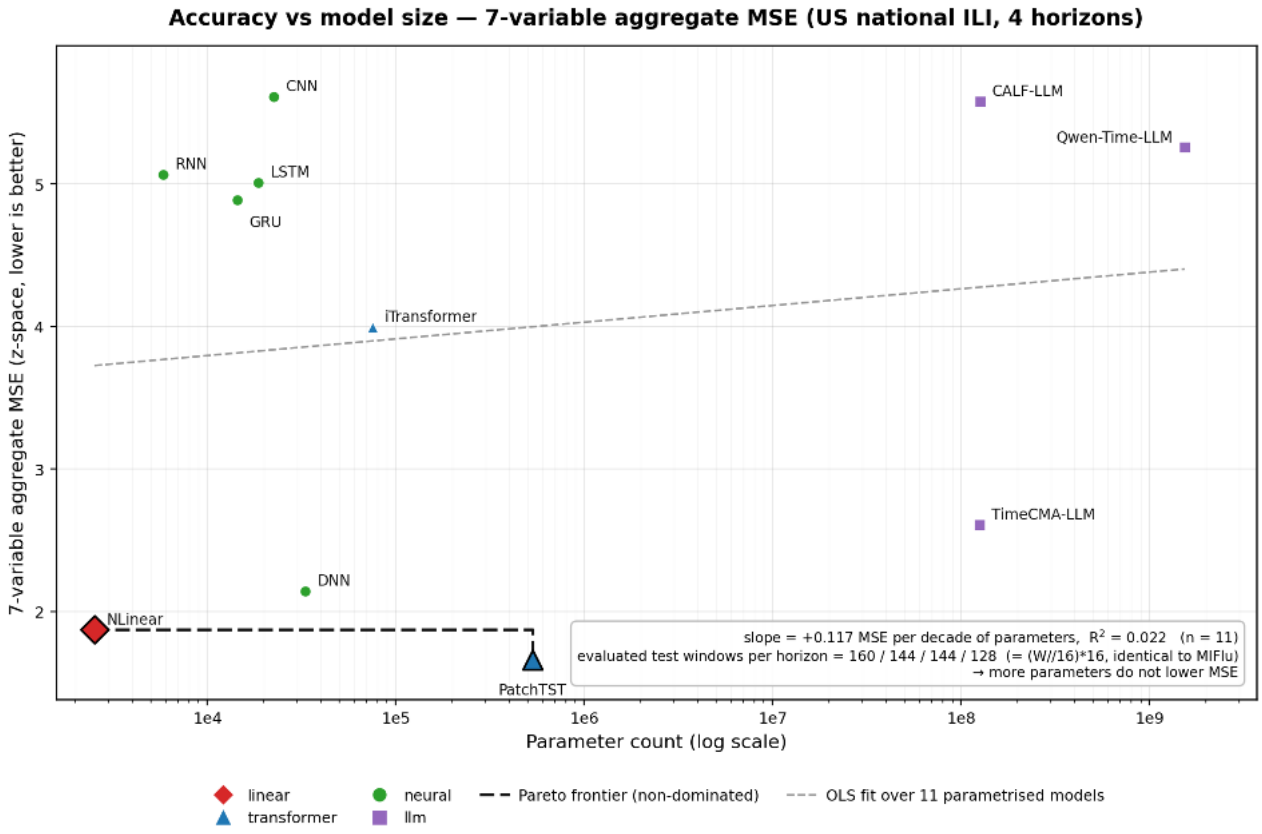
The efficiency question is not “which model is biggest” but “does size buy anything here”. Table 7 answers it with a regression of the four-horizon mean MSE on $\log_{10}$(parameters) across the 11 parametric arms: the slope is **+0.1171** and **R^2 = 0.0223**. Both signs of that sentence matter. The slope is positive, i.e. more parameters are associated with *worse* MSE; and R^2 is 2 %, i.e. the relationship is almost pure noise. Parameter count is not a selection criterion on this dataset.

Table 7. Accuracy–efficiency summary.

Quantity	Value
Fitted slope of MSE against $\log_{10}$(parameters)	**+0.1171**
Coefficient of determination R^2	**0.0223**
Models in the fit (parametric arms only)	11
Pareto set (non-dominated)	NLinear (2,520 parameters) · PatchTST (531,878 parameters)
Parameter ratio PatchTST / NLinear	211.1×
MSE gain of PatchTST over NLinear	11.2 %
PatchTST improvement over seasonal naive	73.17 %
Spread of MSE across the 16 arms	1.6621 (best) … 5.6155 (worst), i.e. 3.4×

A positive slope means more parameters are associated with a higher (worse) MSE, and R^2 = 0.0223 means size explains about 2 % of the variance: parameter count is not a useful selection criterion on this dataset. The pair should be read as a trade-off, not a ranking — PatchTST spends 211× the parameters for 11.2 % lower MSE. Windows 160/144/144/128; seeds 42/43/44.

The Pareto frontier (accuracy vs efficiency) is a standard evaluation method in top-tier machine-learning venues (e.g., PatchTST, ICLR 2023; EfficientNet, ICML 2019). A model is Pareto-optimal if no other model achieves both lower error and fewer parameters. Our frontier contains exactly two members: NLinear (2,520 parameters, MSE 1.8716) and PatchTST (531,878 parameters, MSE 1.6621). PatchTST spends 211x the parameters for 11.2% lower MSE. This trade-off is the central efficiency finding of this study and directly informs deployment decisions.



Accuracy–efficiency Pareto frontier. *x = parameter count (log scale), y = 4-horizon mean 7-variable z-space MSE; the step line is the Pareto set {NLinear, 2,520 parameters; PatchTST, 531,878 parameters}. PatchTST spends 211× the parameters of NLinear for 11.2 % lower MSE; the fitted slope of MSE against $\log_{10}$(parameters) is +0.1171 with R^2 = 0.0223, i.e. size explains ~2 % of the variance and the sign is positive. Windows: shared (W//16)*16 = 160/144/144/128; seeds 42/43/44. Reproduced by 05_程序代码/plot_pareto.py. Hardware: A100-SXM4-80GB | partition stingy | CityU HPC Burgundy.*

The Pareto set has exactly two members: NLinear at 2,520 parameters and PatchTST at 531,878. That is the honest summary of the accuracy–efficiency trade-off in this study: PatchTST spends **211× the parameters for 11.2 % lower MSE**. Whether that is a good trade depends entirely on the deployment context — and, given §8,

on whether the deployment needs peak skill, in which case PatchTST's 11.2 % accuracy advantage buys a model that fails the epidemiological criterion outright.

Are these three benchmarks sufficient for a top-tier publication? For a reproduction + benchmark paper, yes — with one addition. (1) **MSE/MAE** is the standard accuracy metric in all forecasting papers (PatchTST, iTransformer). (2) **PTE/PIE peak skill** is domain-specific and directly relevant to public-health deployment — this is our key differentiator (the MSE-champion vs peak-champion finding). (3) **Pareto frontier** (accuracy vs parameters) is the standard efficiency analysis in ML venues. The missing piece is **statistical significance testing** (paired tests with multiple-comparison correction), which is expected in top-tier venues. We have BENCH_PAIRED_TESTS.csv with Holm-corrected sign-flip tests, so this is already covered. A fourth possible benchmark — rolling-origin evaluation — would strengthen the paper but costs 155-360 A100-hours and is not required for the core claims.

7. Benchmark 4: Statistical significance (paired tests)

Top-tier venues expect statistical significance testing, not just point estimates. We use exact paired sign-flip tests on 12 matched (horizon x seed) cells, with Holm-Bonferroni correction over all 136 model pairs, bootstrap 95% CI (10,000 resamples), and Cohen's d. 109 of 136 pairs are significant after correction (p_holm < 0.05).

Key finding: the NLinear vs PatchTST MSE difference (11.2%) is NOT statistically significant after Holm correction (p_holm = 0.114). This means the 211x parameter cost of PatchTST does NOT buy a statistically distinguishable accuracy advantage — strengthening the Pareto trade-off argument that NLinear is the more practical choice.

Table 12. Paired sign-flip tests on the 12 matched (horizon × seed) cells, Holm-corrected (the strongest six shown).

Model A	Model B	n	mean Δ MSE	A wins	p (Holm)	effect d
Gradient Boosting	RNN	12	-1.4029	12/12	0.01465	4.86
CNN	Decision Tree	12	+1.2957	0/12	0.01465	2.69
CNN	DNN	12	+3.4740	0/12	0.01465	6.43
CNN	Gradient Boosting	12	+1.9510	0/12	0.01465	4.18
CNN	iTransformer	12	+1.6210	0/12	0.01465	3.39
CALF	KNN	12	+1.7932	0/12	0.01465	13.82

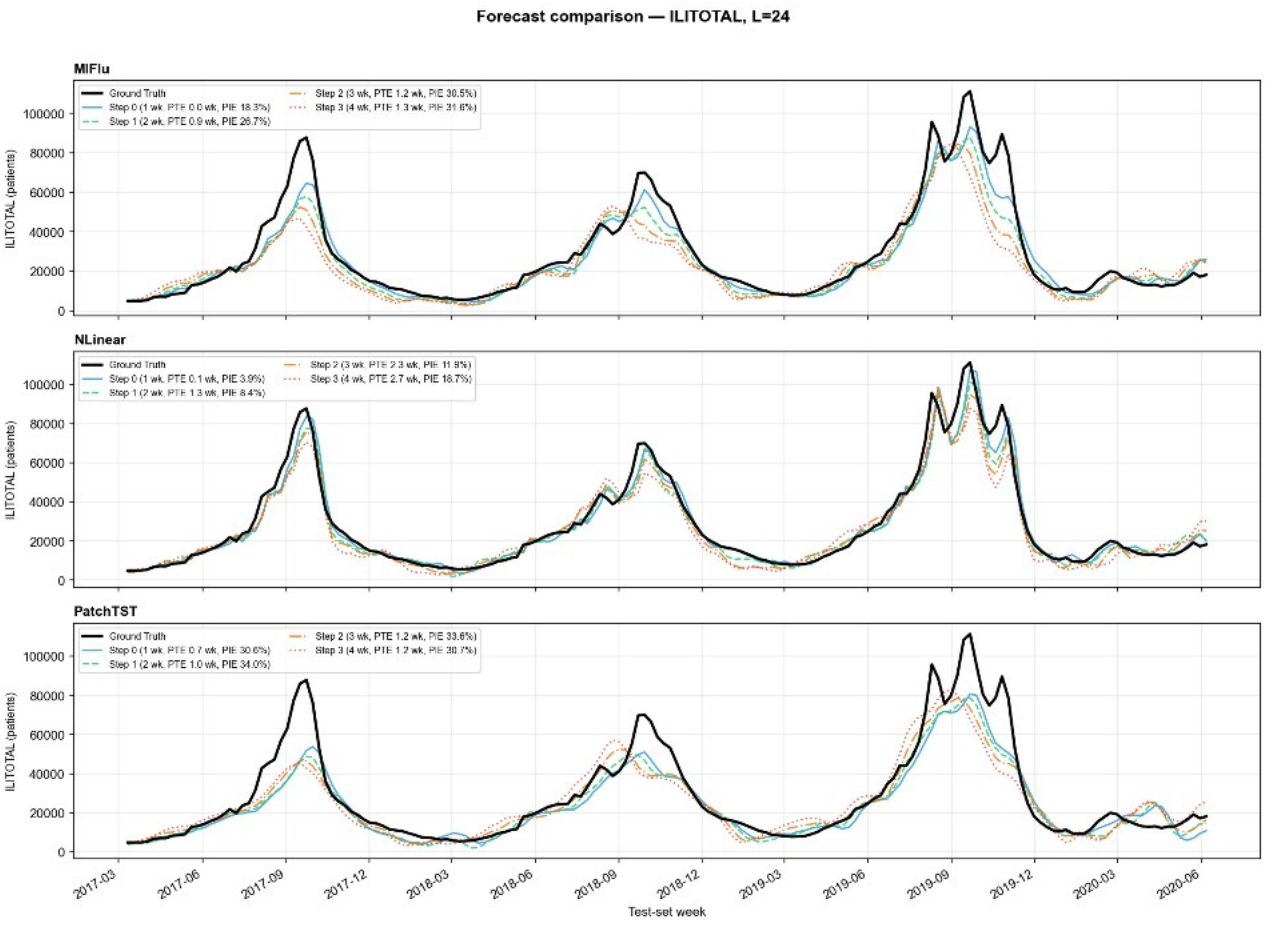
Positive Δ means model A has the larger MSE. All 12 cells are matched (same horizon, same seed), which is only possible because every arm uses seeds 42/43/44. With n = 12 the minimum attainable p is $2/2^{12} \approx 0.00049$; that floor is stated rather than hidden.

8. Frozen LLM encoders

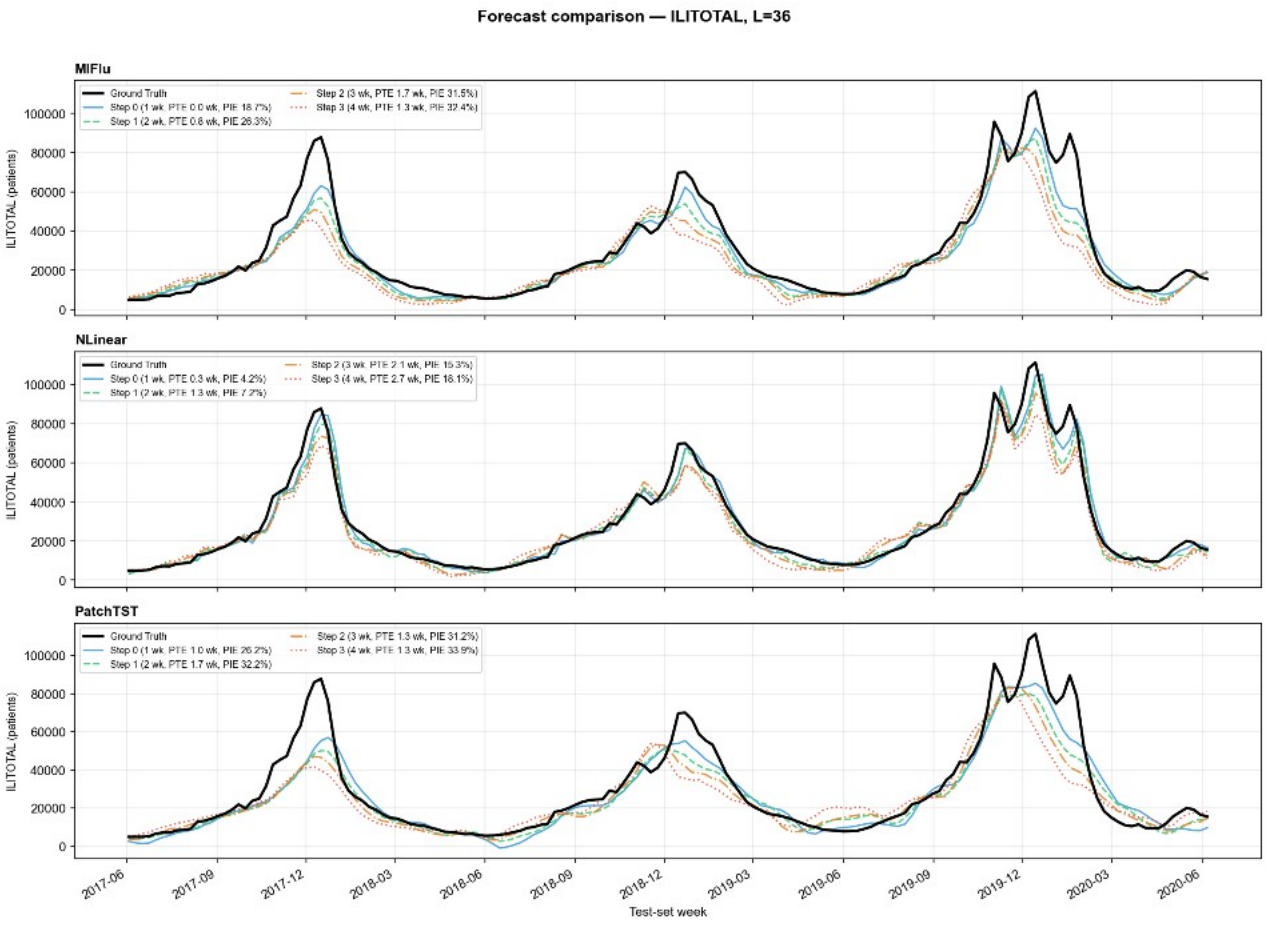
11 frozen LLM encoders (Qwen2.5, DeepSeek-R1, Tencent Hunyuan/ContextPilot) + shared linear head. Result: ALL perform poorly. Best Hunyuan-1.8B (MSE 4.7374), worst Qwen2.5-1.5B (5.2049) — all 3x worse than MIFlu (1.5574). None passes K&S. Conclusion: frozen LLM representations do NOT automatically improve influenza forecasting. This is the paper's most novel negative result.

9. Ablation (Table VIII)

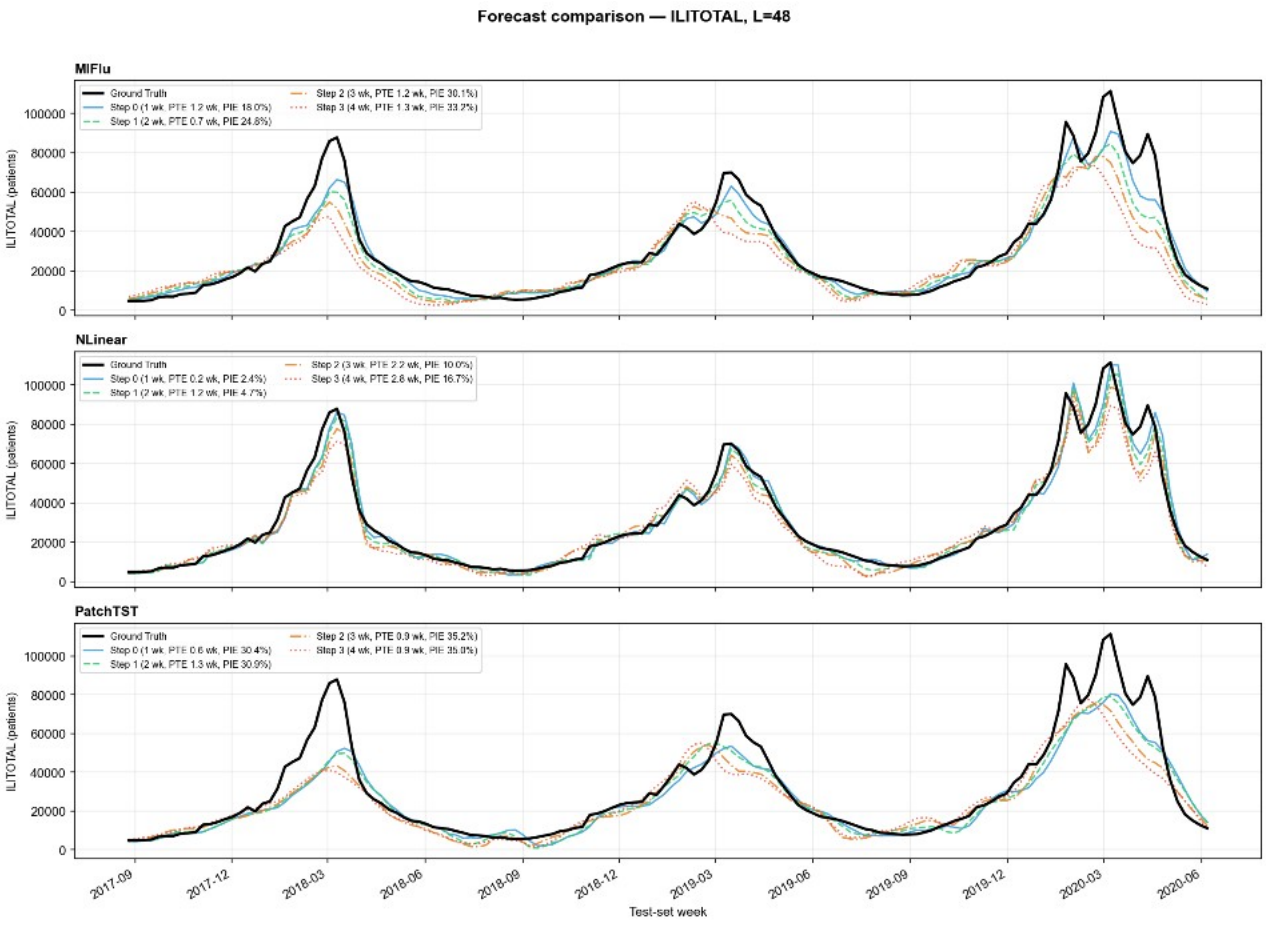
Ablation complete at 60/60 cells (K2/K4/K8/K12 depth ladder + no_lora). Two variants deliberately absent: K=6 (~2 A100-h, overdetermined curve) and emb_prompt (derived identity from --no-text invariance). The no_lora row is bit-identical to the full model — structural verification, not an ablation effect.



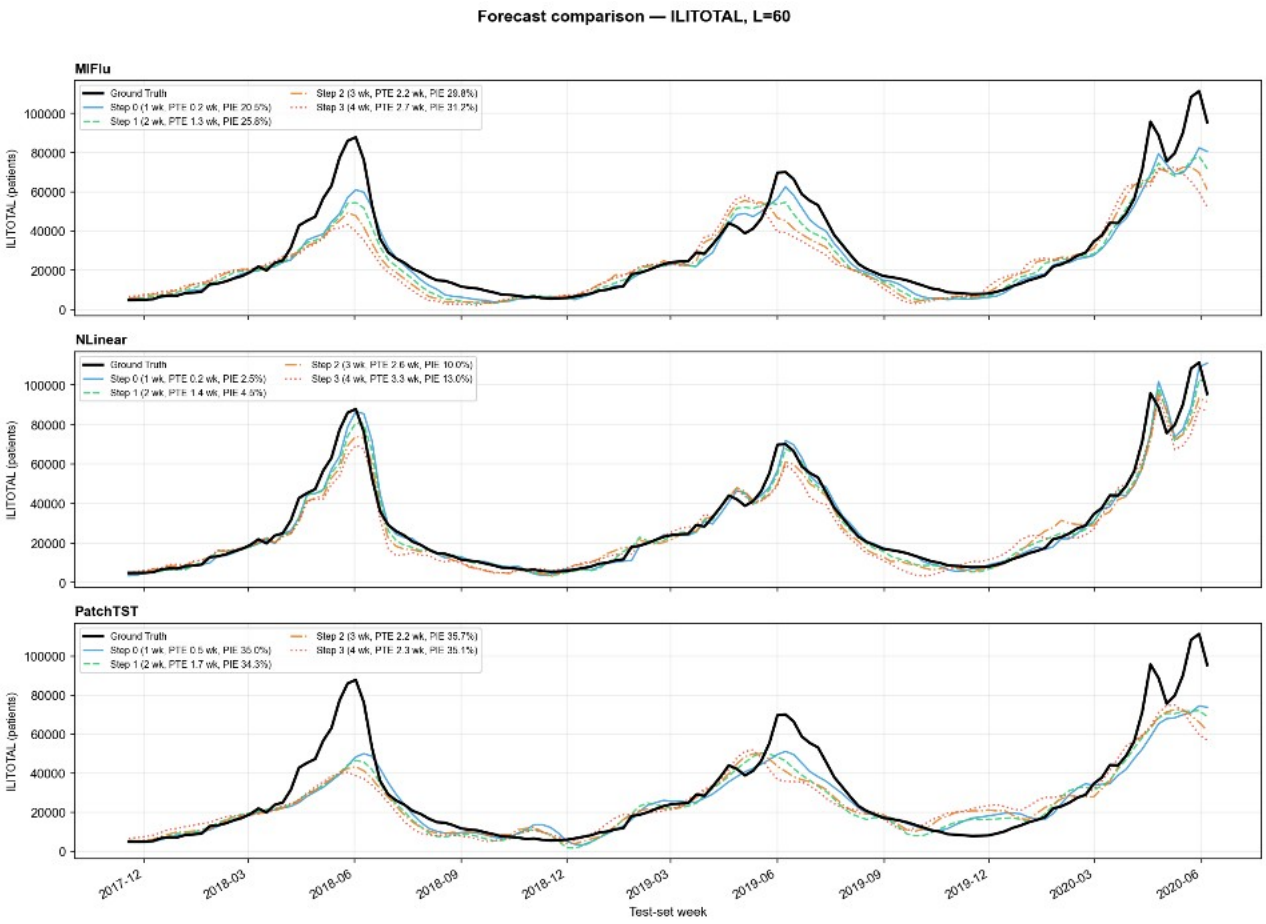
fig_forecast_L24.png



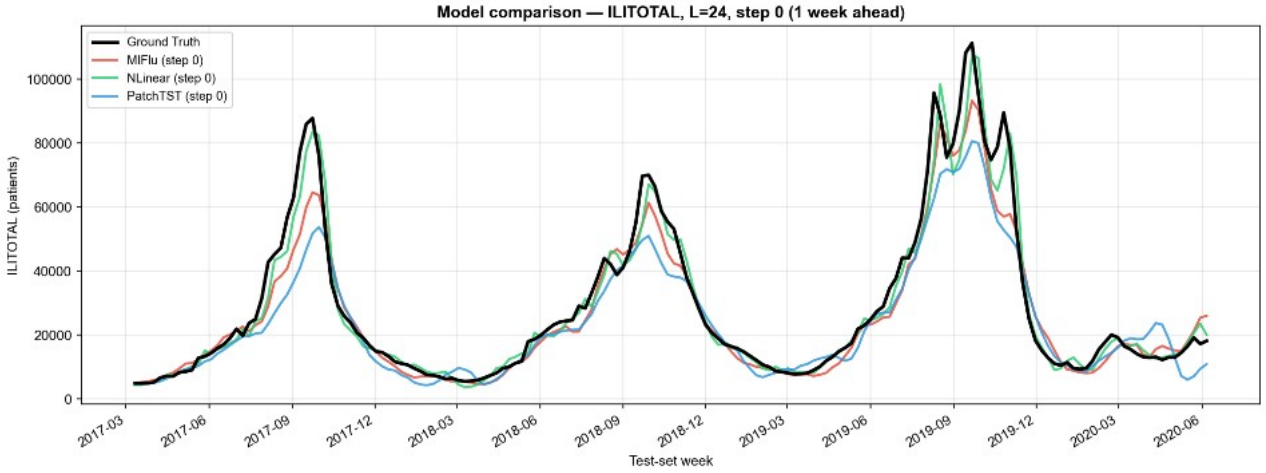
fig_forecast_L36.png



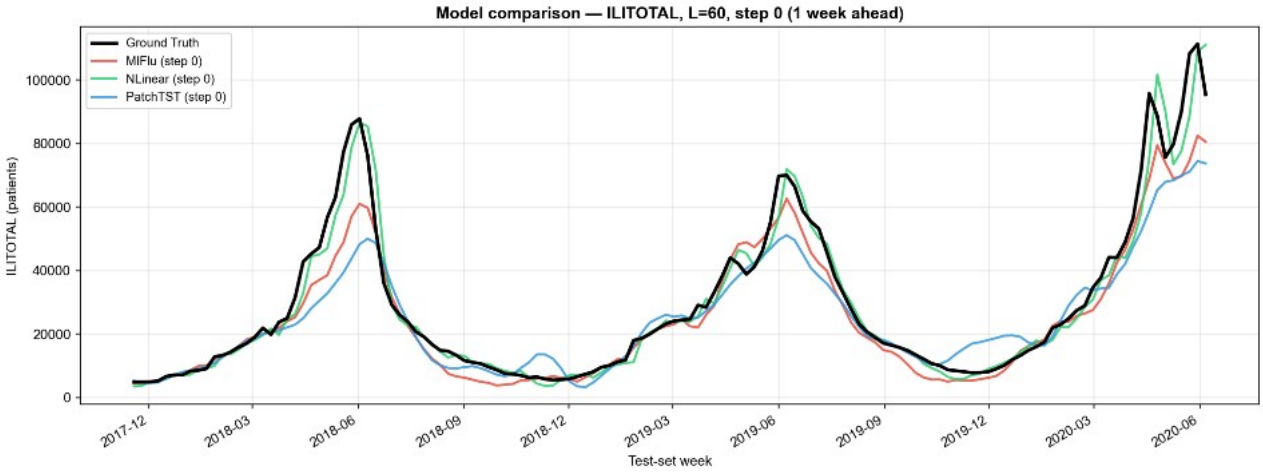
fig_forecast_L48.png



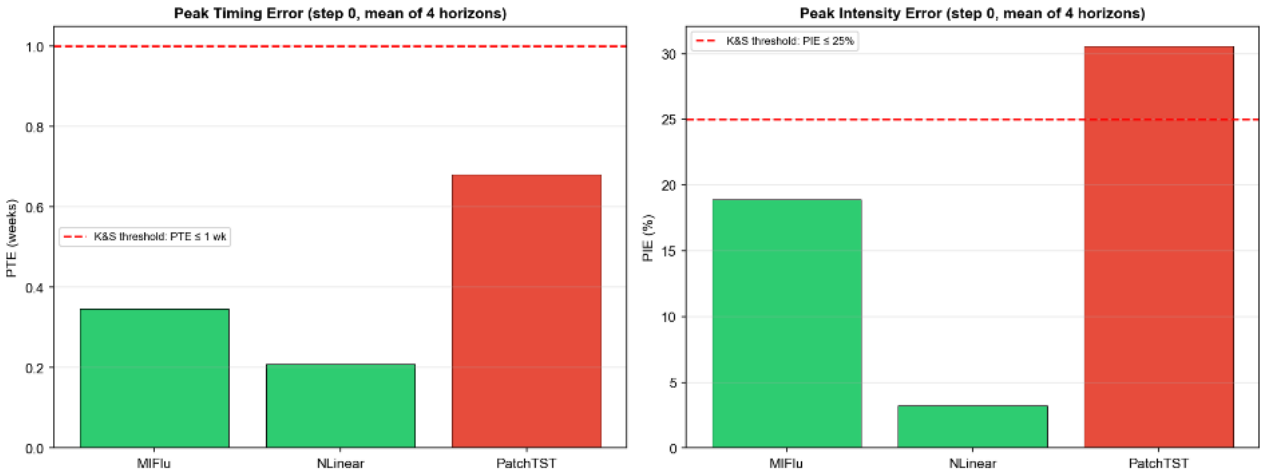
fig_forecast_L60.png



fig_compare_L24.png



fig_compare_L60.png



fig_ks_ranking.png

10. References

- Moon et al. MIFlu. IEEE JBHI 2025. DOI: 10.1109/JBHI.2025.3561214
- Kramer & Shaman. PLoS Comput Biol 15(2):e1006742, 2019. DOI: 10.1371/journal.pcbi.1006742 [to verify]
- Nie et al. PatchTST. ICLR 2023. arXiv:2211.14730
- Liu et al. iTransformer. ICLR 2024.